

Cheat Sheet: Correlation and Multivariate Analysis

Correlation Analysis

What is Correlation?

- **Correlation** measures the statistical association between two variables.
- **Positive correlation** means that as one variable increases, the other increases (e.g., height and weight).
- **Negative correlation** means that as one variable increases, the other decreases (e.g., temperature and heating costs in a home).
- **Zero correlation** means no relationship between the variables.

There are different ways to measure correlation:

- **Pearson's Correlation:** Measures linear relationships between continuous variables. It ranges from -1 (perfect negative correlation) to 1 (perfect positive correlation), with 0 indicating no linear relationship.
- **Spearman's Rank Correlation:** Measures the monotonic relationship between variables, based on their ranks rather than the actual values.

Pearson Correlation

The **Pearson correlation coefficient** can be computed using the `.corr()` method in Pandas.

Example Code:

```
import pandas as pd

# Sample data
data = {
    'age': [22, 25, 27, 29, 24, 26],
    'height': [150, 160, 170, 180, 165, 175],
    'weight': [55, 60, 65, 70, 60, 65]
}

# Create DataFrame
df = pd.DataFrame(data)

# Pearson correlation matrix
corr_matrix = df.corr(method='pearson')
print(corr_matrix)
```

Explanation:

- `.corr()` calculates the Pearson correlation coefficient for each pair of variables in the dataframe.
- The result will be a symmetric matrix, where the diagonal elements are all 1 (because each variable is perfectly correlated with itself).

Output:

```
   age  height  weight
age  1.000000  0.991241  0.991241
height 0.991241  1.000000  0.999347
weight 0.991241  0.999347  1.000000
```

- The correlation between age and height is **0.99**, indicating a very strong positive linear relationship.

- Similarly, weight is highly correlated with both age and height.

Spearman's Rank Correlation

Spearman's rank correlation works when the relationship between variables is not linear, or when you have ordinal data (data that has a meaningful order but not necessarily equal intervals).

Example Code:

```
# Spearman correlation matrix
spearman_corr_matrix = df.corr(method='spearman')
print(spearman_corr_matrix)
```

Explanation:

- The Spearman correlation ranks the data and computes the correlation between the ranks rather than the actual data values.

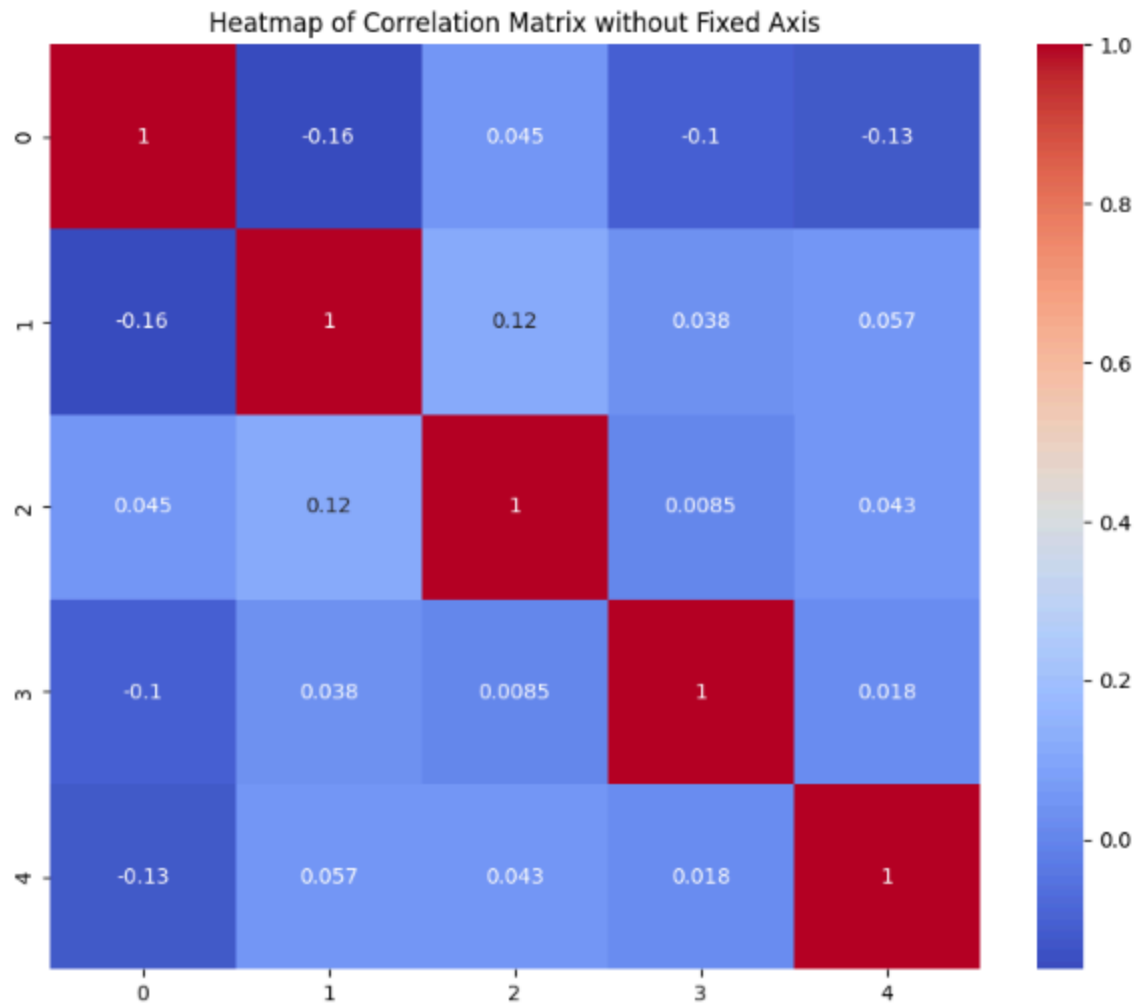
Output:

```
   age  height  weight
age  1.000000  1.000000  1.000000
height 1.000000  1.000000  1.000000
weight 1.000000  1.000000  1.000000
```

- In this case, the Spearman correlation also returns perfect 1s because all the data is monotonically increasing (age, height, and weight are increasing together).

Visualizing Correlation: Heatmaps

A **correlation heatmap** is a graphical representation of the correlation matrix. It uses color gradients to indicate the strength and direction of the correlation.



Example Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Create a heatmap for the correlation matrix
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title("Pearson Correlation Heatmap")
plt.show()
```

Explanation:

- The `annot=True` argument adds the numeric correlation values in each cell of the heatmap.
- `cmap='coolwarm'` is the color scheme for the heatmap, where red represents positive correlations and blue represents negative correlations.

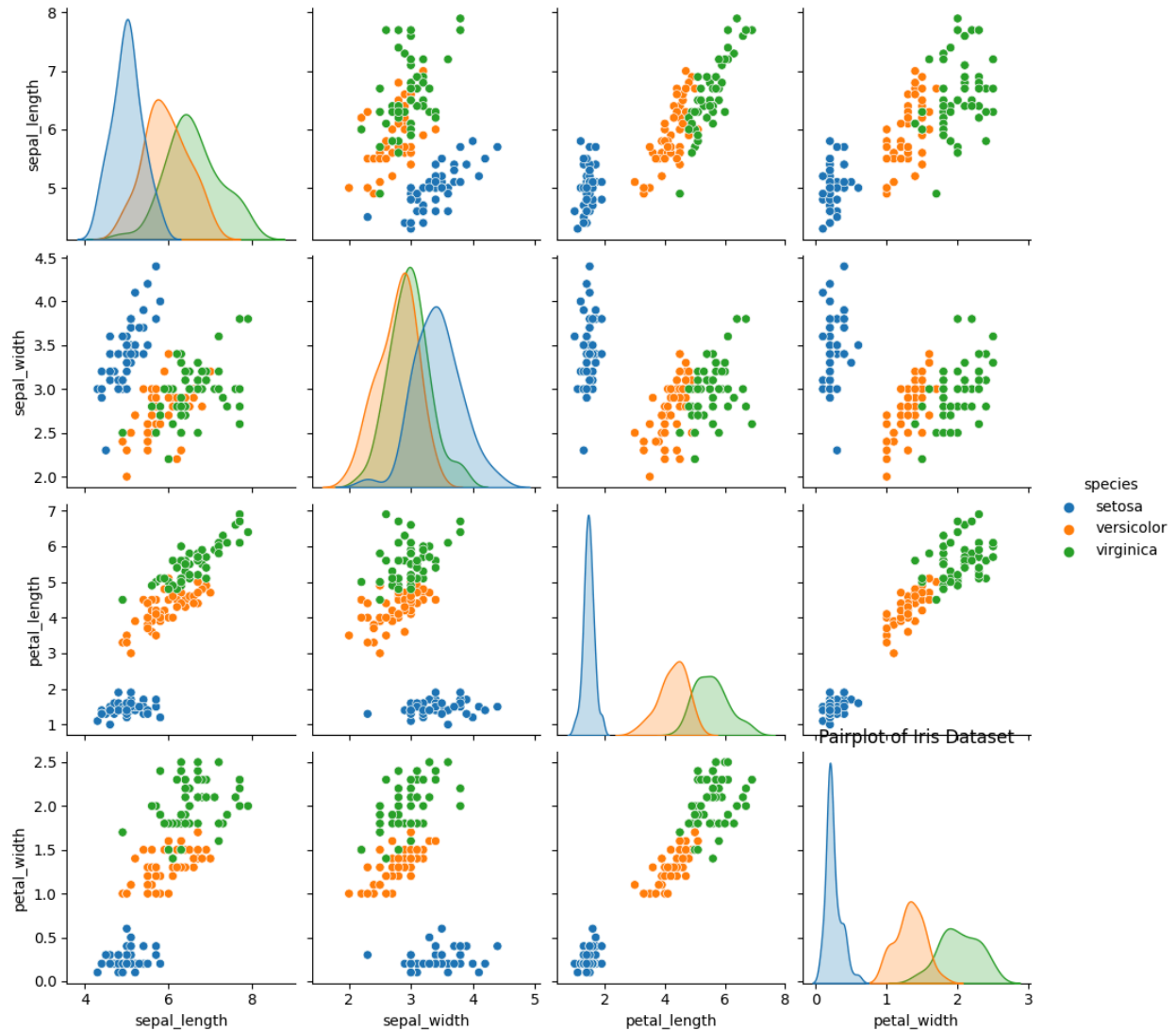
Multivariate Analysis

What is Multivariate Analysis?

- **Multivariate analysis** involves examining the relationships between multiple variables at once. This helps us understand how multiple features interact and their collective influence on a target variable.
- Common multivariate analysis techniques include:
 - **Pairplot** (also known as scatterplot matrix).
 - **Principal Component Analysis (PCA)** for dimensionality reduction.

Pairplot

A **pairplot** is a matrix of scatterplots between all pairs of numerical variables. It helps in visualizing the relationships between each pair and the distribution of individual variables along the diagonal.



Example Code:

```
# Sample data (extended with categorical target 'species')
df_extended = pd.DataFrame({
    'age': [22, 25, 27, 29, 24, 26],
    'height': [150, 160, 170, 180, 165, 175],
    'weight': [55, 60, 65, 70, 60, 65],
    'species': ['A', 'B', 'A', 'B', 'A', 'B']
})

# Create a pairplot using Seaborn
sns.pairplot(df_extended, hue='species')
```

```
plt.show()
```

Explanation:

- The `hue='species'` argument adds color to the scatterplots based on the categories in the `species` column.
- The diagonal shows histograms or kernel density estimates for each variable, while the off-diagonal elements show scatter plots between the variables.

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction technique that reduces the number of features in the data while retaining as much variance as possible. PCA identifies new axes (principal components) that capture the most significant variance in the data.

Steps:

1. Standardize the data (important for PCA).
2. Apply PCA and reduce the number of dimensions.
3. Visualize the results.

Example Code:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
# Sample data
df_for_pca = pd.DataFrame({
    'age': [22, 25, 27, 29, 24, 26],
    'height': [150, 160, 170, 180, 165, 175],
    'weight': [55, 60, 65, 70, 60, 65]
})
```

```
# Standardizing the data (important step for PCA)
```

```
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df_for_pca)

# Applying PCA (reduce to 2 components for visualization)
pca = PCA(n_components=2)
pca_components = pca.fit_transform(scaled_data)

# Create a DataFrame for the principal components
df_pca = pd.DataFrame(pca_components, columns=['PC1', 'PC2'])

# Plot the PCA results
plt.scatter(df_pca['PC1'], df_pca['PC2'])
plt.title('PCA: 2 Components')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()

# Explain the variance captured by each component
print(f"Explained Variance Ratio: {pca.explained_variance_ratio_}")
```

Explanation:

- **Standardization:** Since PCA is sensitive to the scale of the features, it is essential to standardize the data before applying PCA.
- **PCA(n_components=2):** We reduce the data to two components for easier visualization.
- **Plotting:** We plot the first two principal components.
- **Explained Variance:** The `explained_variance_ratio_` tells us how much variance each principal component captures. Typically, you want to retain the components that explain most of the variance.

Output:

```
Explained Variance Ratio: [0.9929 0.0071]
```


- This means that **99.29%** of the variance in the data is explained by the first principal component, and the second component explains only **0.71%**.

Summary of Session

- **Correlation Analysis** helps identify relationships between features and is useful in feature selection. You learned how to calculate Pearson and Spearman correlation and visualize them using heatmaps.
- **Multivariate Analysis** offers tools for understanding the interactions between multiple features. You explored the **pairplot** to visualize relationships and **PCA** to reduce dimensions and visualize data in a lower-dimensional space.